

EngageNet

Progress Update - September 23, 2026

Lado Turmanidze

Supervisors: Dr. Philipp Müller & Beso Mikaberidze

September 23, 2026

The Short Version

What happened since the last meeting

The model was built, it trained, and it produced numbers - but the numbers were bad.

I went looking for *why*, and found six problems. Two of them are not opinions: they are arithmetic. The model literally could not do what we designed it to do.

**All six are fixed, the paper is rewritten, and it has now been retrained.
Best validation CCC went from 0.0227 to 0.4228.**

Today's Agenda

1. **Recap** - what the task is, what the model does
2. **Where we stood** - the numbers that started this
3. **The map** - a small theorem telling us where to look
4. **Problem 1** - the model had no memory
5. **Problem 2** - the modalities never talked to each other
6. **Problem 3** - we predicted the wrong thing
7. **Problem 4** - the loss function had three bugs
8. **Results** - what the retrained model actually does
9. **Since August** - checking our own numbers, and a method that never ran
10. **What we fixed, and what is next**

PART 0: What Is The Task?

The setup

Two people talk over a video link. One is the **expert** (explains something they love), one is the **novice** (listens, asks questions).

Human annotators watch the recording and move a slider to say *how engaged is this person, right now*.

Our job

For **every frame** of video, and for **each of the two people**, predict that slider value.

$$y_t \in [0, 1], \quad 25 \text{ frames per second}$$

A 10-minute session is $\sim 15,000$ numbers per person. We predict all of them.

Recap - The Data We Get

Four feature types $\mathcal{F}$

- ▶ eGeMAPS v2 - acoustic descriptors
- ▶ W2v-BERT 2.0 - learned speech embedding
- ▶ OpenFace2 - facial action units
- ▶ OpenPose - body keypoints

Two roles $\mathcal{R}$

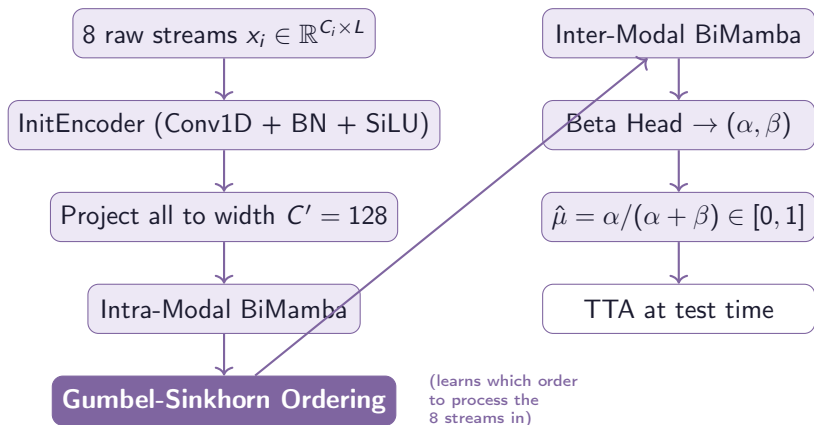
Separate camera and microphone per person, so **every feature type arrives twice**:

$$M = |\mathcal{F}| \times |\mathcal{R}| = 4 \times 2 = 8$$

8 parallel streams into the network.

Remember this $M = 8$. Half of today's talk is about what the model does with these 8 streams.

Recap - Architecture



Recap - What A State-Space Model Actually Does

One line of maths, and it is the whole idea

$$\xi_t = \bar{A}\xi_{t-1} + \bar{B}_t u_t$$

ξ_t is the model's **memory** at step t . Each step it keeps a fraction $\bar{A}$ of what it remembered, and adds in the new input u_t .

Why this shape

$\bar{A} \in (0, 1)$, so old information fades smoothly instead of blowing up. Nice and stable.

Why it is fast

Looks sequential, but the update is *affine*, so we can compute all L steps with an associative scan in $O(\log L)$ depth instead of $O(L)$.

Keep $\bar{A}$ in mind. Problems 1 and 2 are both about this one number.

Recap - Why Beta Instead Of A Plain Number

The model outputs a distribution

Not one number, but two: (α, β) , defining

$$p(y | x) = \text{Beta}(\alpha, \beta), \quad y \in [0, 1]$$

Prediction is the mean $\hat{\mu} = \frac{\alpha}{\alpha + \beta}$.

Why bother

The *spread* tells us how sure the model is:

$$\text{Var}[y] = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}$$

Test-time adaptation uses this to pick which unlabelled frames are safe to learn from.

Beta lives on $[0, 1]$ - exactly the range of engagement. A Gaussian would put probability on -0.3 and 1.4 , which are not things.

PART 1: Where We Stood

What training gave us

- ▶ Best validation CCC: **0.0227**
- ▶ Reached at epoch 2, then flat
- ▶ Correlation $\rho \approx 0.229$
- ▶ 7.2M parameters

What the leaderboard looks like

Official baseline	0.400
2025 winner	0.699
Us	0.02

This is not “needs tuning”. This is “something is structurally wrong”.
So instead of tuning, I went looking for what.

PART 2: A Small Theorem That Told Us Where To Look

Split CCC into two factors

Let $\varphi = \sigma_{\hat{y}}/\sigma_y$ (are the predictions as spread out as the truth?) and $\omega = (\mu_{\hat{y}} - \mu_y)/\sqrt{\sigma_{\hat{y}}\sigma_y}$ (are they centred right?). Then

$$\text{CCC} = \rho \cdot C_b, \quad C_b = \frac{2}{\varphi + \varphi^{-1} + \omega^2}$$

Proposition

$\text{CCC} \leq \rho$, with equality only when $\sigma_{\hat{y}} = \sigma_y$ and $\mu_{\hat{y}} = \mu_y$.

Proof. $\varphi + \varphi^{-1} \geq 2$ for any $\varphi > 0$ (AM-GM), and $\omega^2 \geq 0$. So the denominator of C_b is at least 2, giving $C_b \leq 1$. $\square$

Translation: correlation is a hard ceiling. Rescaling the output can never beat it.

What The Theorem Told Us

Where we were (5 weeks ago)

$$\rho = 0.229, \quad \text{CCC} = 0.21$$

$$\implies C_b = \frac{0.21}{0.229} \approx 0.92$$

Already 92% calibrated - so rescaling the output could buy at most $0.229 - 0.21 = 0.02$.

Where we are now (NoXi, epoch 16)

$$\rho = 0.7951, \quad \text{CCC} = 0.7942$$

$$\implies C_b = \frac{0.7942}{0.7951} = \mathbf{0.9989}$$

Calibration is now essentially perfect. The gap the theorem said was closable, we closed.

PROBLEM 1: The Model Had A Goldfish Memory

How long does an SSM remember?

Stop feeding it input. Then $\xi_{t_0+k} = \bar{A}^k \xi_{t_0}$: memory decays geometrically. Define τ_{eff} as the number of steps until only e^{-1} ($\approx 37\%$) of it survives - the standard time constant of a decaying response.

$$\bar{A}^{\tau_{\text{eff}}} = e^{-1} \implies \tau_{\text{eff}} \ln \bar{A} = -1 \implies \tau_{\text{eff}} = \frac{-1}{\ln \bar{A}}$$

Now substitute $\bar{A} = \exp(\Delta A)$, so $\ln \bar{A} = \Delta A$. Since $A < 0$ by construction, write $A = -|A|$, giving $\ln \bar{A} = -\Delta|A|$ and therefore

$$\tau_{\text{eff}} = \frac{-1}{-\Delta|A|} = \frac{1}{\Delta|A|}$$

Here $\Delta > 0$ is the **discretisation timestep**: how much continuous time one discrete step represents. Small Δ = fine steps = slow forgetting = long memory.

Problem 1 - Why 58 ms Is A Disaster

The mismatch

Engagement changes over **seconds**. Our model could see **58 milliseconds**.

That is about one and a half frames. It is like judging whether someone is enjoying a film by looking at a single blink.

Reference Mamba initialises Δ between 10^{-3} and 10^{-1} . Ours was 0.693 - **seven times larger than their maximum**.

And it gets worse

$A_{\log} = 0$ means *every one* of the $N = 16$ state coordinates got the **same** decay rate.

We paid for 16 memories at different timescales and received 16 identical copies of one very short memory.

Problem 1 - The Fix

S4D-Real initialisation

Index channels by $d \in \{1, \dots, C'\}$ and, within each channel, state coordinates by $n \in \{0, \dots, N-1\}$ (we use $C' = 128$ channels, $N = 16$ coordinates each). Give every coordinate its own decay rate:

$$A_{\log}[d, n] = \ln(n+1) \implies a_{d,n} = -(n+1) \in \{-1, \dots, -16\}$$

and spread Δ log-uniformly over $[10^{-3}, 10^{-1}]$ per channel.

Before

$\Delta = 0.693$ and $|A| = 1$, for every d and n .

$$\tau_{\text{eff}} = \frac{1}{0.693 \times 1} = 1.4 \text{ frames}$$

One timescale, 58 ms.

After (measured at init)

Δ alone spans $\tau_{\text{eff}} = 1/\Delta \in [10, 1000]$ frames; the $|A| \in [1, 16]$ ladder divides that by up to 16, so coordinates cover ≈ 0.6 to 1000 frames.

At 25 Hz one frame is 40 ms, so 10 frames = 0.4 s and 1000 frames = **40 s**.

PROBLEM 2: The Modalities Never Talked To Each Other

The whole point of the fusion stage

Take the 8 processed streams and let them share information - so the model can notice “the face says bored *but* the voice says excited”.

What we found

We had **two different architectures**: one described in the paper, one actually running in the code.

They were different from each other, and both were broken - in different ways.

Problem 2a - What The Paper Described

Lay the 8 streams end to end, scan along that line

Each stream is $C' = 128$ channels wide, so the line is $MC' = 1024$ positions long and **each modality occupies a block of 128**.

The telephone game problem

Information from one modality must survive $C' = 128$ steps to reach the next, and each step keeps a fraction $\bar{A}$. That fraction is the *same* number from Problem 1: at initialisation $\bar{A} = \exp(\Delta A) = \exp(-0.693 \times 1) = 0.5$. So

$$\bar{A}^{C'} = 0.5^{128} \approx 3 \times 10^{-39}$$

and the smallest number float32 can represent is $\approx 1.2 \times 10^{-38}$.

The message does not arrive faint. It arrives as exactly zero.

Like whispering down a line of 128 people where each one halves the volume.

Problem 2b - What The Code Actually Did

Concatenate the modalities as *features*, scan along time

Different design. Here modalities *do* mix - through a big dense layer $W \in \mathbb{R}^{MC' \times MC'}$ that touches everything at once. So fusion happens.

But two things go wrong.

The ordering did nothing

Our whole Gumbel-Sinkhorn machinery learns a permutation P . But

$$W(P \otimes I_{C'}) U = W' U, \quad W' := W(P \otimes I_{C'})$$

A dense layer can **absorb** any reordering into its own weights. The permutation was decorative.

It ate the whole model

$D = MC' = 1024$, and the block has five 1024×1024 matrices:

$$5.36\text{M of } 7.20\text{M} = 74\%$$

Three quarters of our parameters, on 31 training sessions.

Problem 2 - The Fix: Make Modality The Sequence

Stop unpacking modalities into channels

Before: modality i was split into its 128 individual channels, laid end to end, so it occupied *positions* $[(i-1)C', iC')$ of a 1024-long sequence. **After:** at each time step t , modality i 's whole C' -dimensional vector $\tilde{u}_i^{(t)}$ is *one* position. The sequence is the $M = 8$ modalities:

$$H^{(t)} = \text{BiMamba}_{\text{modality}}(\tilde{u}_1^{(t)}, \dots, \tilde{u}_M^{(t)}), \quad t = 1, \dots, L'$$

So neighbouring modalities go from $C' = 128$ steps apart to **1 step apart** - not by shortening the path, but by no longer inserting 127 irrelevant positions into it.

What happened to P ?

Nothing was removed. P was absorbable only because a *dense layer* followed it. Now an *SSM scan* follows it, and scans are order-sensitive - so P decides which modality supplies context to which.

And it got cheaper

Fusion block width drops from MC' to C' :

$$\Theta(M^2 C'^2) \rightarrow \Theta(C'^2)$$

5.36M $\rightarrow$ 96,640 parameters.

PROBLEM 3: We Predicted The Wrong Thing

What we were doing

The challenge asks for **each person's** engagement. We were averaging the two into one number and predicting that:

$$\bar{y}_t = \frac{1}{2}(y_t^e + y_t^n)$$

Why averaging destroys signal

With both annotations having variance σ^2 and correlation r :

$$\text{Var}(\bar{y}) = \frac{1}{4}[\sigma^2 + \sigma^2 + 2r\sigma^2] = \frac{\sigma^2(1+r)}{2}$$

If the two people's engagement moves *oppositely*, averaging flattens the curve towards a straight line.

Problem 3 - How Bad Is It? The Corpus Paper Measured It

“Variance kept” means what fraction of the wiggle survives averaging

From the previous slide, $\text{Var}(\bar{y}) = \sigma^2(1 + r)/2$. Dividing by σ^2 , the fraction kept is $(1 + r)/2$. Engagement that never moves is unpredictable by construction, so this is signal we destroy before training starts.

Expert-novice correlation r

Language	r	$(1 + r)/2$
German	−0.19	41%
English	−0.07	47%
French	−0.05	48%
Japanese	+0.51	76%

(Funk et al., 2024, Sec. 5.3.4)

Three of four are negative

When the two participants move *oppositely*, their average is flatter than either one. On German sessions only 41% of the variance survives.

CCC is normalised by target variance, so a flattened target cannot be un-flattened later.

Japanese is the exception; the paper links it to a stronger tendency toward interpersonal harmony.

Problem 3 - The Fix (Easier Than Expected)

No new machinery needed

The two people were **never mixed on the way in** - their recordings enter as separate streams and stay separately indexed the whole way through.

So: attach **one output head per role** instead of one shared head. Each unimodal head is now trained against the person whose stream it reads.

Bonus 1

This is what the challenge actually asked for. We were solving a different problem than the one being scored.

Bonus 2

Gender is per person. With one averaged output, a session where the two differ had *no valid group label* - we were silently dropping those sessions from the fairness metric.

PROBLEM 4: The Loss Function Had Three Bugs

The old loss, in full

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{\text{NLL}}(\alpha, \beta, y) + \lambda_u \sum_{i=1}^M \mathcal{L}_{\text{NLL}}(\alpha_i, \beta_i, y), \quad \lambda_u = 0.5$$

1. It rewarded being *calibrated*, but we are scored on being *correlated*
2. It paid most attention to the frames the model already knew
3. The 8 helper heads outweighed the main head by 4x

Bonus defect: nothing in it ever asked the model to be fair.

Problem 4 - Where This Is Going

One loss became four terms. In plain words:

Term	Asks the model to	Fixes
1 – $\widehat{CCC}$	put frames in the <i>right order</i>	bug 4a
Beta NLL, reweighted	get each frame's <i>distribution</i> right	bug 4b
Per-modality NLL / M	keep every stream useful on its own	bug 4c
Squared CDD	treat groups equally at equal truth	bug 4d

The intuition

1. The first term asks: does the whole story rise and fall in the right places?
2. The second term asks: is this frame's answer sensible?
3. The third stops the eight helper branches from going lazy and free-riding on the fused head.
4. The fourth says: whatever you predict, predict it the same way for everyone.

Bug 4a - Calibrated Is Not The Same As Correlated

What the likelihood asks

“At *this* frame, is your predicted distribution right?”

One frame at a time. Never compares frames to each other.

What CCC asks

“Across *all* frames, do your predictions go up and down in the same pattern as the truth?”

And by Part 2, that is the only thing that can raise our score.

Fix: add the metric itself to the loss

$$\mathcal{L} += \lambda_c (1 - \widehat{\text{CCC}})$$

Every ingredient of CCC (two means, two variances, one covariance) is a smooth function of the predictions, so we can just differentiate it.

Bug 4b - The Loss Ignored The Hard Frames

Differentiate the Beta log-likelihood w.r.t. the mean

Writing $\mu = \alpha/(\alpha + \beta)$, $\kappa = \alpha + \beta$ (so κ = confidence), and using $\partial\alpha/\partial\mu = \kappa$, $\partial\beta/\partial\mu = -\kappa$, the two $\psi(\alpha+\beta)$ terms cancel and leave

$$\frac{\partial(-\ln p)}{\partial\mu} = \kappa \left[\ln \frac{1-y}{y} + \psi(\mu\kappa) - \psi((1-\mu)\kappa) \right] \propto \kappa$$

Read that again

The correction is multiplied by the model's own confidence. Frames it is *already sure about* produce the biggest updates; uncertain frames produce almost none.

It is a teacher who only helps the students who already understand.

Bug 4b - The Fix: One Weight Per Frame

Every frame gets its own weight

A window is 250 frames. For every frame t the model outputs its own (α_t, β_t) , so every frame has its own confidence $\kappa_t = \alpha_t + \beta_t$. The loss is a sum over frames; we multiply each frame's term by a weight:

$$\mathcal{L}_{\text{NLL}}^w = \frac{1}{T} \sum_{t=1}^T w_t \cdot (-\ln p(y_t | \alpha_t, \beta_t)), \quad w_t = \kappa_t^{-\beta_w}$$

Frame t used to pull $\propto \kappa_t$. Now it pulls $\propto \kappa_t \cdot \kappa_t^{-\beta_w} = \kappa_t^{1-\beta_w}$.

Before ($\beta_w = 0$)

A sure frame ($\kappa = 100$) and an unsure one ($\kappa = 4$) pull $100 : 4$ - the sure one **25** $\times$ harder.

After ($\beta_w = 0.5$)

They pull $\sqrt{100} : \sqrt{4} = 10 : 2$ - only **5** $\times$.
 $\beta_w = 1$ would make every frame equal; 0.5 is the middle ground.

Adapted from β -NLL (Seitzer et al., ICLR 2022), which does the same for Gaussian outputs.

Bug 4b - When Are The Weights Computed, And Why stop_grad?

Every training step, in this order

1. Forward pass: the model predicts (α_t, β_t) for every frame, so we know every κ_t .
2. Compute $w_t = \kappa_t^{-0.5}$ and **freeze** it - from here on it is a plain number.
3. Backward pass: each w_t is a constant multiplier on its frame's gradient.
4. Update the parameters. Next step κ_t has changed, so w_t is recomputed from scratch.

The weights are never learned. They are re-read from the model's current confidence every step.

Without stop_grad

w_t depends on the parameters too, so it becomes part of what is optimised. The model can lower the loss by *changing its confidence* to shrink the weight, not by predicting better.

With it

The weights change *how hard* each frame pulls, not *where* the pulling stops: the best (α, β) is the same as for the plain likelihood.

Bug 4c - The Helper Heads Were Running The Show

The arithmetic

$M = 8$ helper heads, each weighted $\lambda_u = 0.5$:

$$8 \times 0.5 = 4$$

versus 1 for the main head.

And at test time we **only use the main head**.

The fix

Divide by M :

$$\frac{\lambda_u}{M} \sum_{i=1}^M \mathcal{L}_{\text{NLL}}(\alpha_i, \beta_i, y^{r(i)})$$

Now the helpers contribute 0.5 total, not 4, and adding more modalities does not silently change the balance.

80% of the gradient was coming from parts of the network we throw away at evaluation.

Bug 4d - We Measured Fairness But Never Asked For It

Do equally engaged women and men get the same prediction?

Plain group averages are the wrong test: if one group really is more engaged, a perfect model *should* predict higher for it. So compare like with like:

1. Sort the batch's frames by **true** engagement, cut into n_b equal-count bins.
2. In each bin, average women's and men's predictions separately; take the gap.
3. $\mathcal{L}_{\text{fair}} = \sum_b |\mathcal{B}_b| \text{gap}_b^2 / \sum_b |\mathcal{B}_b|$ (squared, so signs cannot cancel)

Example (illustrative)

Truth ≈ 0.6 : women predicted 0.66, men 0.60. Gap 0.06, penalty 0.0036; the gradient nudges women down, men up, in that bin.

Honest caveat

Zero on a batch with only one gender. A mixed-gender pair puts both groups in the same window.

The New Loss, All Together

$$\mathcal{L} = \underbrace{\frac{1}{|\mathcal{R}|} \sum_r \mathcal{L}_{\text{NLL}}^w(\alpha^r, \beta^r, y^r)}_{\text{per-frame fit, rebalanced}} + \underbrace{\lambda_c(1 - \widehat{\text{CCC}})}_{\text{ordering}} + \underbrace{\frac{\lambda_u}{M} \sum_i \mathcal{L}_{\text{NLL}}^w(\alpha_i, \beta_i, y^{r(i)})}_{\text{per-modality}} + \underbrace{\lambda_f \mathcal{L}_{\text{fair}}}_{\text{fairness}}$$

Term	Asks the model to...	Weight
Beta NLL (reweighted)	get each frame's distribution right	$1, \beta_w = 0.5$
$1 - \widehat{\text{CCC}}$	rank frames correctly	$\lambda_c = 1.0$
Per-modality NLL	keep every stream individually useful	$\lambda_u/M = 0.0625$
Squared CDD	treat groups equally at equal truth	$\lambda_f = 0.05$

What Changed, In Numbers

	Before	After
Memory span τ_{eff}	1.4 frames (58 ms)	0.6–1000 frames (up to 40 s)
Distinct decay rates	1	16
Cross-modal distance	$C' = 128$ steps	1 step
Fusion block parameters	5,360,000 (74%)	96,640 (5.0%)
Total parameters	7,197,898	2,064,844
Output heads	1 shared	2 (one per person)
Loss terms	2	4
Permutation actually used	no	yes
Best val CCC, NoXi+J	0.0227	0.4228
Best val CCC, NoXi	—	0.7942

The NoXi column is new: we had never trained on it at all.

PART 3: Three Runs, In Order

	Corpus	Best CCC	Epoch	What we learned
Run 1	NoXi+J	0.4228	15 of 25	fixes work; wrong corpus
Run 2	NoXi	0.7500	7 of 12	patience 5 stopped on noise
Run 3	NoXi	0.7942	16 of 31	patience 15, $\lambda_c = 1.0$

Run 3, best epoch

val CCC	0.7942
ρ	0.7951
expert CCC	+0.700
novice CCC	+0.615
CDD _G	-0.0146

Why run 3 beat run 2

Raising λ_c from 0.5 to 1.0 pushed C_b from 0.92 to 0.9989.

Patience 15 let τ anneal further - it needs ~ 45 epochs to reach $\tau_{\min}$, and run 2 died at 12.

The Number That Actually Counts Is Lower

Two ways to score the same model

Training loop 0.7942

evaluate.py **0.6987**

NoXi val, 10 sessions, 577,176 frames.

Trust the lower one

The training loop pools *windows*: with 50% overlap every interior frame is counted twice, and zero-padded tails count as real.

evaluate.py reassembles full sessions by overlap-add, exactly as the challenge does. Averaging overlaps shrinks variance, which costs C_b .

0.6987 on NoXi val, against a 2026 baseline of 0.55 on NoXi test.
Fairness under proper evaluation: $CDD_G = +0.006$, CDD_L within ± 0.027 .

PART 4: Since August - Did We Keep The Right Checkpoint?

Run 3, all saved checkpoints

Epoch	NoXi val CCC
10	0.6732
16 (kept)	0.6987
20	0.6267
30	0.6472

`evaluate.py`, `overlap-add`. Only every 10th epoch was saved.

What this answers

- ▶ The optimistic metric still picked the right one: of the four we have, epoch 16 is best.
- ▶ After epoch 16 the model gets *worse*. “Train longer so τ fully anneals” moves down the list.
- ▶ Checkpoints a few epochs apart differ by up to 0.07. Any change smaller than that needs repeated runs before we believe it.

A Model Trained On One Corpus Is Nearly Blind On The Other

Every model, scored on every corpus

	Scored on NoXi	Scored on NoXi+J
Trained on NoXi (Run 3)	0.6987	0.0922
Trained on NoXi+J (Run 1)	0.1429	0.4337

NoXi: English, French, German. NoXi+J: Japanese, Chinese. Same features, same architecture, same task.

The off-diagonal is near zero

What a model learns about engagement on one set of people and languages barely transfers to the other.

This is exactly the gap test-time adaptation was designed to close.

Side note: on NoXi+J the honest score (0.4337) is *higher* than the training loop's (0.4228). The overlap-add gap is not a fixed discount.

Immediate consequence

Use one model per corpus. Mean over the two corpora goes from 0.3955 to **0.5662**.

No new training, just the right model on the right data.

TTA - How It Is Supposed To Work

The idea

At test time there are no labels. But the model has a **fused** head that sees all 8 streams, and 8 **single-stream** heads that each see one. The fused head acts as a *teacher*: on unlabelled test windows, we nudge the model so the single-stream heads agree with it.

Only where teaching is safe and useful

Every head reports its uncertainty u (the Beta variance from Part 0). Teach on window i only if

- ▶ the teacher is **sure**: its u is among the *lowest* 30% in the batch, and
- ▶ the students are **unsure**: their average u is among the *highest* 30%.

Written out, with $q_p :=$ the value that a fraction p of the batch falls below:

$$\text{keep}_i = [u_i^{\text{multi}} < q_{0.3}(u^{\text{multi}})] \wedge [u_i^{\text{uni}} > q_{0.7}(u^{\text{uni}})]$$

TTA Was Written, Wired In - And Never Fired

With a batch of 10 windows

$q_{0.3}$ sits around the 3rd-lowest teacher uncertainty, so roughly the 3 surest windows pass the first test; $q_{0.7}$ sits around the 7th, so roughly the 3 most confused pass the second. Windows in both groups are kept.

Our inference feeds one window at a time

A batch of one. The 30% point of a single number is that number, $q_{0.3}(\{u\}) = u$, and $u < u$ is never true. **Every window is rejected, so the adaptation step is never called.**

Two more problems behind it

Even when triggered, it adapts on *every* window in the batch, not only the kept ones; and it outputs predictions from *before* the update.

What it means for our numbers

Every CCC in this talk is **without** TTA. The method in our paper's title has, in effect, never run - and the previous slides say we need it.

What We Fixed

In the model

- ▶ Memory span: 58 ms $\rightarrow$ up to 40 s (P1)
- ▶ Modalities now actually fuse, the ordering matters, 7.2M $\rightarrow$ 2.1M parameters (P2)
- ▶ One prediction per person, as the challenge asks (P3)
- ▶ Loss: ranking, hard frames, balance, fairness (P4)

In the pipeline

- ▶ Broken NaN entries in NoXi label files repaired
- ▶ Scoring through the real challenge pipeline, not the training loop
- ▶ Checkpoint choice re-checked on the real scores
- ▶ Found why TTA never fired

NoXi+J val CCC: 0.0227 $\rightarrow$ 0.4337. NoXi val CCC: 0.6987.
Fairness gaps (CDD) within ± 0.03 on both corpora.

In order

1. **Make TTA work, then run NoXi $\rightarrow$ NoXi+J** - it is the method in the title, and cross-corpus transfer (0.09) is exactly its job.
2. **Submit per-corpus test predictions** - both models' test outputs are written, not yet checked.
3. **Retrain NoXi+J with Run 3 settings** - running now; it was trained with older settings.
4. **One model on both corpora** - does more data beat two specialists?
5. **Repeat runs with different seeds** - checkpoints differ by up to 0.07, so we need to know how much of any gain is noise.
6. **Ablate the four loss terms** - which of them actually earns its place?

Leaderboard context: best NoXi (base) test is 0.82; the 2026 baseline is 0.55.